

CSc 445 — Homework #5 Solution

Network Flow and Matching in Bipartite Graphs

1. **26.1-6.** (from CLRS) Given a flow network $G = (V, E)$, let f_1 and f_2 be functions from $V \times V$ to $\mathbf{R}$. The **flow sum** $f_1 + f_2$ is the function from $V \times V$ to $\mathbf{R}$ defined by

$$(f_1 + f_2)(u, v) = f_1(u, v) + f_2(u, v)$$

for all $u, v \in V$. If f_1 and f_2 are flows in G , which of the three flow properties must the flow sum satisfy, and which might it violate?

Answer:

The flow sum satisfies skew symmetry since for all $u, v \in V$ it is true that

$$\begin{aligned} (f_1 + f_2)(u, v) &= f_1(u, v) + f_2(u, v) && \text{(definition of flow sum)} \\ &= -f_1(v, u) - f_2(v, u) && \text{(skew symmetry of } f_1 \text{ and } f_2) \\ &= -(f_1(v, u) + f_2(v, u)) && \text{(distributive law)} \\ &= -(f_1 + f_2)(v, u) && \text{(definition of flow sum).} \end{aligned}$$

The flow sum also satisfies flow conservation since for all $u, v \in V - \{s, t\}$ one has

$$\begin{aligned} \sum_{v \in V} (f_1 + f_2)(u, v) &= \sum_{v \in V} (f_1(u, v) + f_2(u, v)) && \text{(definition of flow sum)} \\ &= \sum_{v \in V} f_1(u, v) + \sum_{v \in V} f_2(u, v) && \text{(distributive law)} \\ &= 0 + 0 && \text{(flow conservation of } f_1 \text{ and } f_2) \\ &= 0. \end{aligned}$$

However, flow sum does not satisfy capacity constraint since for all $u, v \in V$ one also has

$$\begin{aligned} (f_1 + f_2)(u, v) &= f_1(u, v) + f_2(u, v) && \text{(definition of flow sum)} \\ &\leq c(u, v) + c(u, v) && \text{(capacity constraint of } f_1 \text{ and } f_2) \\ &= 2c(u, v) \\ &\not\leq c(u, v) && \text{(unless } c(u, v) = 0). \end{aligned}$$

2. **25.3-6** (from CLRS) Professor Michener claims that there is no need to create a new source vertex in line 1 of JOHNSON. He claims that instead we can just use $G' = G$ and let s be any vertex in $V[G]$. Give an example of a weighted, directed graph G for which incorporating the professor's idea into JOHNSON causes incorrect answers. Then show that if G is strongly connected (every vertex is reachable from every other vertex), the results returned by JOHNSON with the professor's modification are correct.

Answer:

The problem with Professor Michener's approach comes from the possibility that not every vertex v is not necessarily reachable from the source vertex s that is picked. For instance, if G is the graph with two vertices $V = \{s, v\}$ with only one directed edge from

v to s , i.e. $E = \{(v, s)\}$. Further suppose that $w(v, s) = 1$. When BELLMAN-FORD is run on G using s as the source vertex, $h(s) = \delta(s, s) = 0$, but $h(v) = \delta(s, v) = \infty$ (i.e there is no directed path from s to v). Then when reweighting the edge weight for (v, s) , we get $\widehat{w}(v, s) = w(v, s) + h(v) - h(s) = 1 + \infty - 0 = \infty$. This gives $\widehat{\delta}(v, s) = \infty$ when running Dijkstra's algorithm from v . However, when computing d_{vs} from $\widehat{\delta}(v, s)$ we get

$$d_{vs} = \widehat{\delta}(v, s) + h(s) - h(v) = \infty + 0 - \infty = \infty - \infty,$$

which is an undefined value, hence, uncomputable.

However, if the graph is strongly connected, i.e. there is a directed path from every vertex $v \in V$ to every other vertex $w \in V - \{v\}$, then the above possibility cannot arise. As such, $-\infty < h(v) = \delta(s, v) < \infty$, i.e. $h(v)$ is some finite real number. Since $h(v) \leq h(u) + w(u, v)$ (from the triangle inequality), and since $-\infty < w(u, v) < \infty$ for all $(u, v) \in E$, these together imply that $0 \geq \widehat{w}(u, v) = w(u, v) + h(u) - h(v) < \infty$, for all $(u, v) \in E$.

As such, the conditions to use Johnson's algorithm, which rely on the fact that reweighting does not alter shortest paths, and that all the adjusted edge weights $\widehat{w}(u, v)$ are non-negative, hold. Additionally, since G is strongly connected, $0 < \widehat{\delta}(u, v) < \infty$ must also be the case after running Dijkstra's algorithm from each vertex $u \in V$. Then $d_{uv} = \widehat{\delta}(u, v) - h(v) + h(u)$ is also finite, and hence, correct.

3. Assume f is a flow on the network G , and p is a path from s to t in the residual network G_f . Let $c_f(p) = \min\{c_f(u, v) : (u, v) \in p\}$. We define a new function $f'(u, v)$ by setting for every pair $u, v \in V$ to be as follows:

$$\begin{aligned} f'(u, v) &= f(u, v) + c_f(p) && \text{if } (u, v) \in p, \\ f'(u, v) &= f(u, v) - c_f(p) && \text{if } (v, u) \in p, \text{ and} \\ f'(u, v) &= f(u, v) && \text{otherwise.} \end{aligned}$$

Prove that f' is a flow on G . Note that there are a few constraints that you need to prove.

Answer:

This problem is really just a restatement of **Corollary 26.4** on page 654 of the text, whose proof relies on **Lemma 26.2**, which is proved on page 653 of the text, and **Lemma 26.3**, whose proof is left as an exercise. **Lemma 26.3** defines a function $f_p : V \times V \rightarrow \mathbf{R}$ for an augmenting path p in G_f as follows:

$$\begin{aligned} f_p(u, v) &= c_f(p) && \text{if } (u, v) \in p, \\ f_p(u, v) &= -c_f(p) && \text{if } (v, u) \in p, \text{ and} \\ f_p(u, v) &= 0 && \text{otherwise.} \end{aligned}$$

where $c_f(p)$ is defined as above, which means that $f_p(u, v) = f'(u, v) - f(u, v)$.

Lemma 26.3 contends that $f_p(u, v)$ is a flow in G_f with value $|f_p| = c_f(p) > 0$. To see that this is the case, we need to show that $f_p(u, v)$ satisfies the three flow conditions as follows:

- **Capacity constraint:** Since $c_f(u, v)$ is the minimum of the capacity of the edges along the path p from s to t if (u, v) or (v, u) is on p , and 0 otherwise, no capacities can be exceeded.

- **Skew symmetry:** If (u, v) is on p , $f_p(u, v) = c_f(p) = -(-c_f(p)) = -f_p(v, u)$. Similarly, if (v, u) is on p , $f_p(v, u) = -c_f(p) = -f_p(v, u)$. Finally, if $f_p(u, v) = 0$, $f_p(v, u) = -0 = -f_p(u, v)$.
- **Flow conservation:** Let $u \in V - \{s, t\}$. If u is on $p = s \rightsquigarrow x \rightarrow u \rightarrow y \rightsquigarrow t$ (where both $s = x$ and $y = t$ are possibilities) then

$$\sum_{v \in V} f_p(u, v) = f_p(u, x) + f_p(u, y) = -c_f(p) + c_f(p) = 0.$$

Otherwise, if u is not on p , then $\sum_{v \in V} f_p(u, v) = 0$ by definition.

Moreover, if $p = s \rightarrow x \rightsquigarrow t$ (where $x = t$ is a possibility), then

$$|f_p| = \sum_{v \in V} f_p(s, v) = f_p(s, x) = c_f(p) > 0$$

since p is a some augmenting path whose minimum capacity must be greater than 0.

Now we can apply **Lemma 26.2** on page 653, which states that if f is a flow in G and for any flow such as f_p in G_f (the residual network of G induced by f), then the flow sum of $f + f_p$ is a flow in G with value $|f + f_p| = |f| + |f_p|$. Translating this to our problem where $f' = f + f_p$, this with **Lemma 26.3** that we just proved, immediately yields f' being a flow in G with the value

$$|f'| = |f| + |f_p| = |f| + c_f(p) > |f|.$$

Alternatively, one could prove f' is a flow directly, but this approach explicitly shows how $f_p = f' - f$ is a positive flow in G_f , and the motivation behind determining whether f' is a flow with value greater than the original flow f .

4. Let $G = (A \cup B, E)$ be a bipartite graph (this means that the set of vertices is $A \cup B$ such that $A \cap B = \emptyset$), and every edge of E connects a vertex of A to a vertex of B . Assume further that for every $u \in A, v \in B$, the $(u, v) \in E$.

Find a largest-cardinality matching in G by generating a network-flow problem (in the way described in class) and then find a maximum flow using Edmonds-Karp algorithm. Describe every stage of the algorithm.

Answer:

Here add two vertices s and t to form $G' = (V', E')$, so that $V' = A \cup B \cup \{s, t\}$ and

$$E' = E \cup \{(s, u) : u \in A\} \cup \{(v, t) : v \in B\}$$

Furthermore, we set the capacity of all edges to 1 and will run Edmonds-Karp algorithm to determine a maximum flow.

Since there is an edge between every pair (u, v) of vertices in $u \in A$ and $v \in B$, we can without loss of generality label the vertices in A and B by the order in which they will be encountered by the Edmonds-Karp algorithm relative to the other vertices in A and B respectively. As such, we can denote $A = \{a_1, a_2, \dots, a_k\}$ and $B = \{b_1, b_2, \dots, b_\ell\}$ where $k = |A|$ and $\ell = |B|$ and a_i (or b_i) and is encountered before a_j (or b_j) if and only if $i < j$.

Every augmenting path p found by Edmonds-Karp on G' using BFS has length 3 where $p = s \rightarrow a_i \rightarrow b_j \rightarrow t$, for $a_i \in A$ and $b_j \in B$ for some $1 \leq i \leq k$ and $1 \leq j \leq \ell$. This stems from the fact that every vertex in A is adjacent to every vertex in B , so once an edge is used as part of an augmenting path, it will never contribute again to any other

augmenting path produced by Edmonds-Karp algorithm on G' . If this were not the case, then consider the first shortest augmenting path $p' = s \rightarrow a_i \rightsquigarrow b_j \rightarrow t$ of length greater than 3 that Edmonds-Karp generates. Since $p = s \rightarrow a_i \rightarrow b_j \rightarrow t$ cannot be an augmenting path, (a_i, b_j) must already be in some other augmenting path, which must be of length 3 by the assumption that p' is the **first** path of length greater than 3. However, the only such path of length 3 is precisely $s \rightarrow a_i \rightarrow b_j \rightarrow t$, which gives a contradiction.

Hence, once a vertex $u \in A$ is used in some augmenting path, then the edge (s, u) must be used, which prevents any more augmenting paths through u since all augmenting paths are of length 3. Likewise, once a vertex $v \in B$ is used in some augmenting path, then the edge (v, t) must be used, which prevents any more augmenting paths through v . This means that once any vertex in $A \cap B$ is used in an augmenting path, it can never be used by any other augmenting path, so the set of augmenting paths produced by Edmonds-Karp must be precisely,

$$\begin{aligned} p_1 &= s \rightarrow a_1 \rightarrow b_1 \rightarrow t, \\ p_2 &= s \rightarrow a_2 \rightarrow b_2 \rightarrow t, \\ &\vdots \\ p_m &= s \rightarrow a_m \rightarrow b_m \rightarrow t, \end{aligned}$$

where $m = \min\{k, \ell\} = \min\{|A|, |B|\}$.

The largest-cardinality matching produced by Edmonds-Karp on G' is then the set of edges $\{(a_i, b_i) : 1 \leq i \leq m\}$.

The stages of the algorithm are then as follows

1. Construct the graph G' as defined above.
2. Run Edmonds-Karp algorithm on G' .
3. Extract the set of augmenting paths used by Edmonds-Karp running on G' .
4. Extract the set of edges of the largest-cardinality matching of size $m = \min\{|A|, |B|\}$, by deleting the first and last edge from each of the m augmenting paths.

Stage **1** can be done in $O(|E|)$ time. Stages **3** and **4** can be done in $O(\min\{|A|, |B|\})$ time. Finally, stage **2** can be done in $O(\min\{|A|, |B|\}|E|)$ time since exactly $m = \min\{|A|, |B|\}$ flow augmentations are performed on G' , rather than the $O(|V|)$ flow augmentations performed in the normal worst case for Edmonds-Karp on a “standard” bipartite graph.

5. Give an upper bound on the running time of Edmonds-Karp algorithm for the graph described in the previous question. Note that because of the special structure of the graph (i.e. every $u \in A, v \in B$ are connected), the actual running time is slightly better than the one obtained for a “standard” bipartite graph even though the **asymptotic running time is the same**.

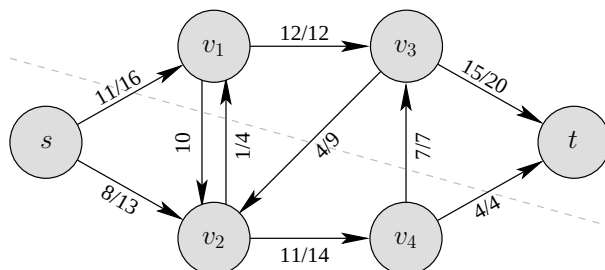
Answer:

The upper bound of $O(\min\{|A|, |B|\}|E|)$ for the running time (along with the argument for why this holds) is given at the end in the previous solution. For a “standard” bipartite graph in which not every edge exists between A and B , the running time would be slightly longer since not every augmenting path is of length 3.

6. **26.2-1** (from CLRS) In Figure 26.1(b), what is the flow across the cut $(\{s, v_2, v_4\}, \{v_1, v_3, t\})$? What is the capacity of this cut?

Answer:

The given cut $C = (S, T)$ where $S = \{s, v_2, v_4\}$ and $T = \{v_1, v_3, t\}$ is illustrated with a gray dashed line in the figure below.



The net flow across this cut is

$$\begin{aligned} f(S, T) &= f(s, v_1) + f(v_2, v_1) - f(v_3, v_2) + f(v_4, v_3) + f(v_4, t) \\ &= 11 + 1 - 4 + 7 + 4 \\ &= 19, \end{aligned}$$

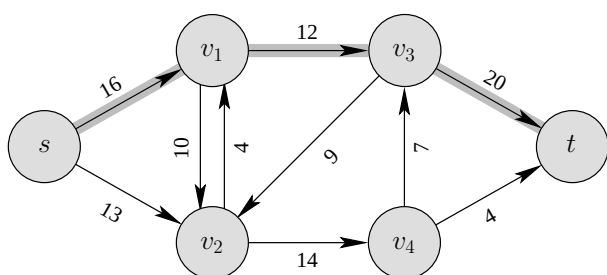
and its capacity is

$$\begin{aligned} c(S, T) &= c(s, v_1) + c(v_2, v_1) + c(v_4, v_3) + c(v_4, t) \\ &= 16 + 4 + 7 + 4 \\ &= 31. \end{aligned}$$

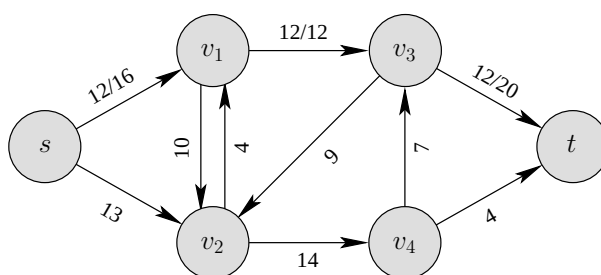
7. **26.2-2** (from CLRS) Show the execution of the Edmonds-Karp algorithm on the flow network of Figure 26.1(a).

Answer:

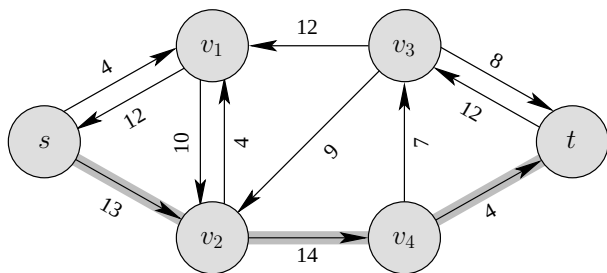
The figures below illustrate the sequence of execution of Edmonds-Karp algorithm on the graph G for each successive iteration of the **while** loop.



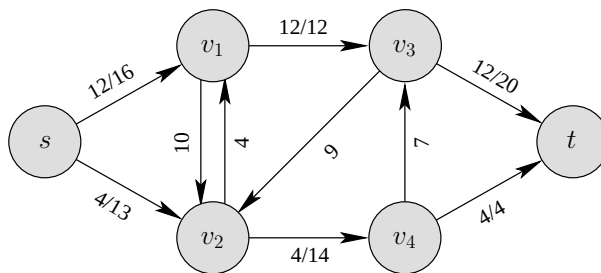
(a)



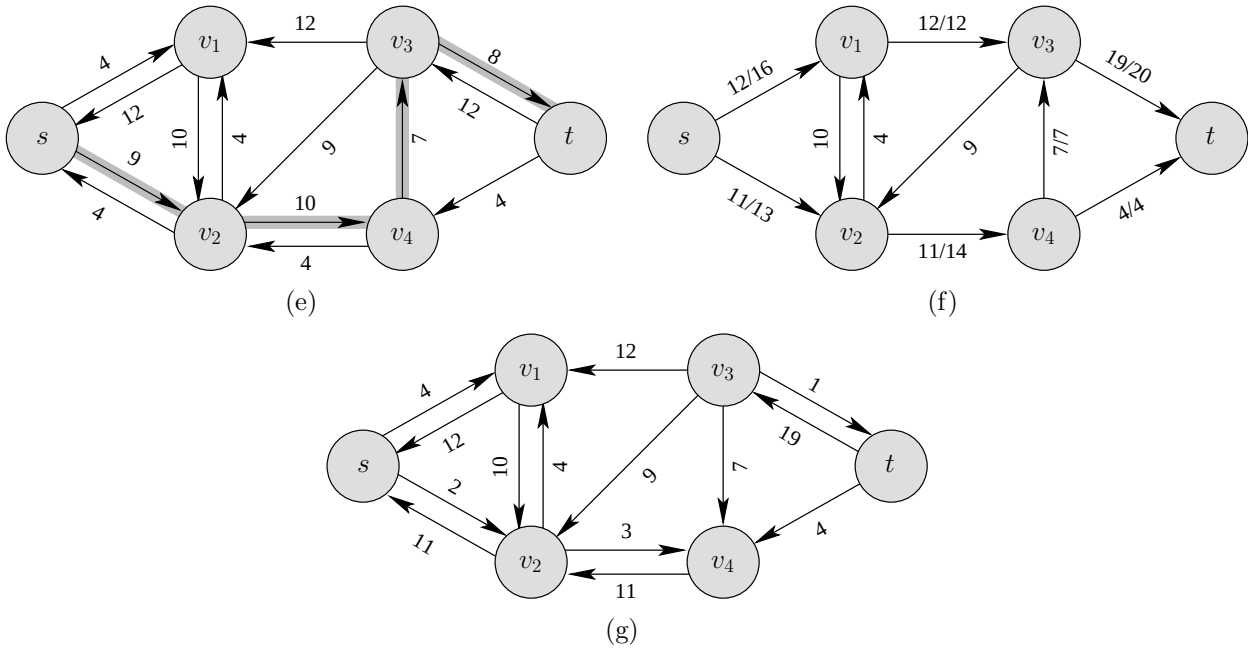
(b)



(c)



(d)



The left side of each row, i.e. (a), (c), (e), are the residual graph networks G_f with the augmenting path p selected by Edmonds-Karp algorithm. The right side of each row, i.e. (b), (d), and (f), shows the new flow f that results in adding f_p to f . The final residual flow is given in (g), which has no augmenting paths.

8. **26.2-4** (from CLRS) Prove that for any pair of vertices u and v and any capacity and flow functions c and f , we have $c_f(u, v) + c_f(v, u) = c(u, v) + c(v, u)$.

Answer:

The proof for this claim falls easily from the definition of $c_f(u, v)$ and the skew symmetry of $f(u, v)$, i.e. for all $u, v \in V$ it is true that

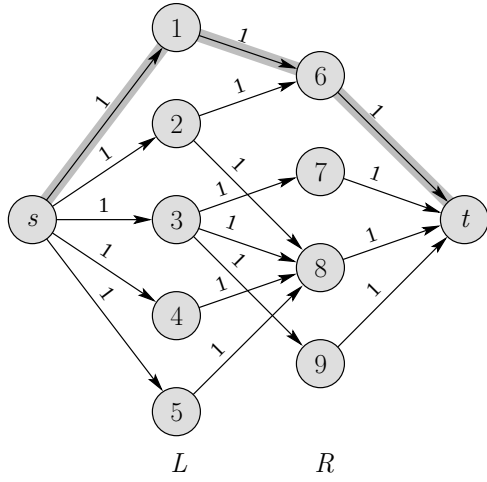
$$\begin{aligned}
 c_f(u, v) + c_f(v, u) &= c(u, v) - f(u, v) + c(v, u) - f(v, u) \\
 &\quad \text{(by definition of } c_f(u, v)) \\
 &= c(u, v) + c(v, u) - f(u, v) - f(v, u) \\
 &= c(u, v) + c(v, u) - f(u, v) + f(u, v) \\
 &\quad \text{(by skew symmetry of } f(u, v) = -f(v, u)) \\
 &= c(u, v) + c(v, u).
 \end{aligned}$$

9. **26.3-1** (from CLRS) Run Ford-Fulkerson algorithm on the flow network in Figure 26.8(b) and show the residual network after each flow augmentation. Number the vertices in L and top to bottom from 1 to 5 and in R top to bottom from 6 to 9. For each iteration, pick the augmenting path that is lexicographically smallest.

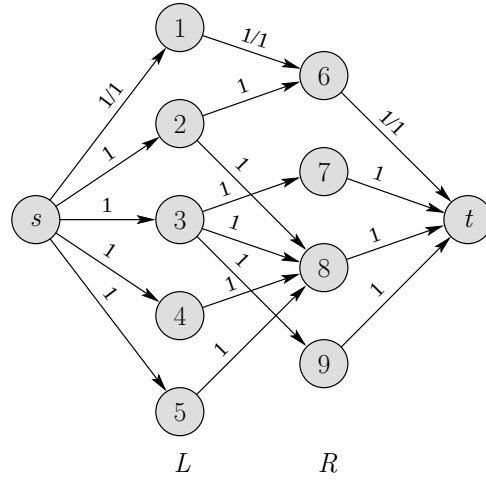
Answer:

The figures on the next page contain each successive iteration of the **while** loop when running Ford-Fulkerson on the graph augmenting the nodes in lexicographic order. The left side of each row, i.e. (a), (c), (e), are the residual graph networks G_f with

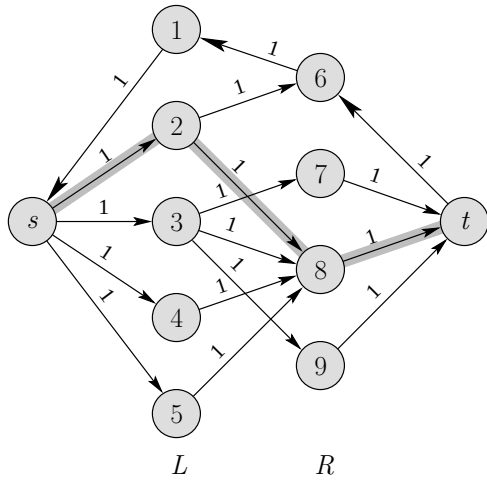
the augmenting path p selected by lexicographic order. The right side of each row, i.e. (b), (d), and (f), shows the new flow f that results in adding f_p to f .



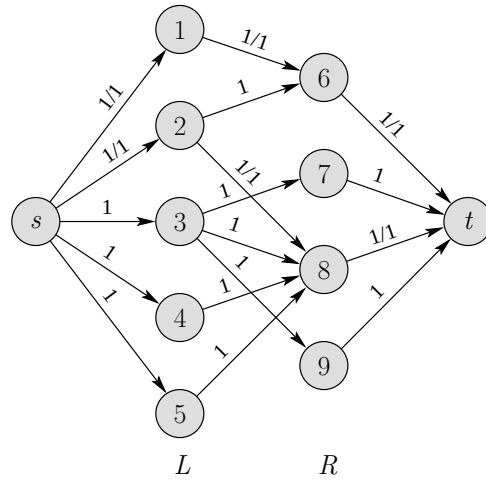
(a)



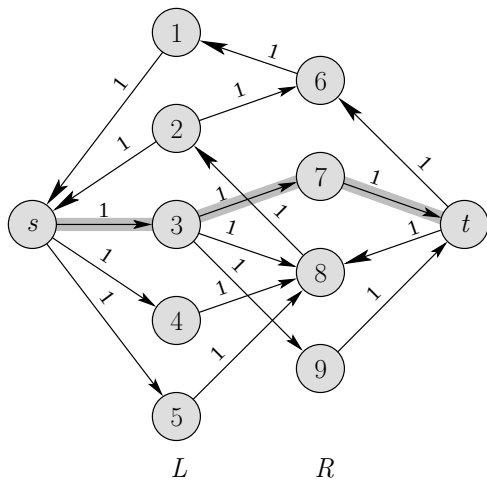
(b)



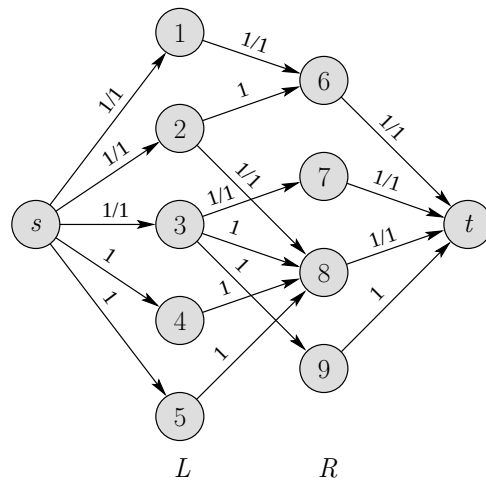
(c)



(d)

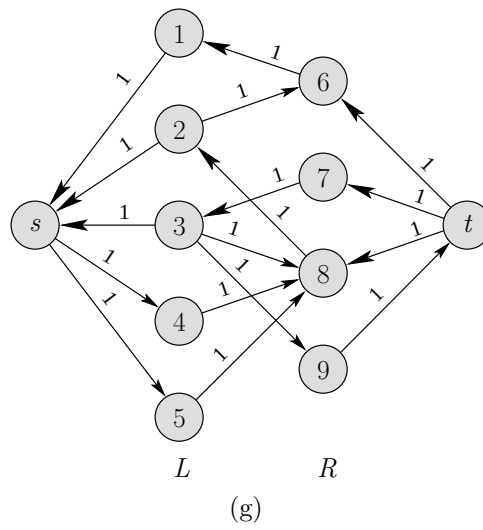


(e)



(f)

The final residual flow is given in (g) below, which has no remaining augmenting paths.



The maximum matching selected by running Ford-Fulkerson using this lexicographic order was $(1, 6)$, $(2, 8)$, $(3, 7)$.